

EquiSplit is a full-stack expense-sharing platform built to demonstrate backend and full-stack engineering ability.

The end product should let users:

1. create an account
2. log in securely
3. create expense groups
4. invite or add members to groups
5. add expenses paid by one user and split across multiple users
6. calculate who owes whom
7. minimize debts into the fewest reasonable settlement transactions
8. mark debts as settled
9. view expense history and current balances in a React dashboard
10. run against a deployed backend and deployed database

The main resume story is:

I built a production-style full-stack financial ledger system with authentication, PostgreSQL transactions, relational schema design, debt optimization logic, API testing, CI/CD, Docker deployment, and a React frontend.

That is the important part.

WebSockets, Redis, Flutter, multi-currency, and fancy analytics are **stretch goals**, not MVP requirements.

Final project scope

Must-have MVP

Frontend

- React
- TypeScript
- Vite
- Tailwind or simple CSS
- Login page
- Register page
- Dashboard
- Group list
- Group detail page
- Add expense form

- Expense history
- Balance summary
- Optimized settlement view
- Settle up action

Backend

- Node.js
- TypeScript
- Express or Fastify
- REST API
- JWT authentication
- Password hashing
- PostgreSQL with Neon
- Prisma or Drizzle ORM
- Zod request validation
- Proper error handling
- Transactional expense creation
- Debt minimization algorithm

Database

Tables:

- users
- groups
- group_members
- expenses
- expense_splits
- settlements
- balance_snapshots or computed balance views
- refresh_tokens, optional
- audit_logs, optional

Deployment

- Backend on Google Cloud Run
- Frontend on Vercel
- Database on Neon
- Dockerfile for backend
- Environment variables
- GitHub repo
- README
- Architecture diagram

Testing

- Unit tests for debt algorithm
 - API tests for auth/groups/expenses
 - At least basic database integration tests
 - GitHub Actions running type checks and tests
-

Stretch goals

Only add these after the core project works.

1. WebSockets for real-time balance updates
 2. Redis Pub/Sub for scalable WebSocket fanout
 3. Multi-currency expense support
 4. Email invitations
 5. Flutter mobile client
 6. Better analytics dashboard
 7. Audit logs
 8. Admin/debug panel
 9. Rate limiting
 10. OpenAPI/Swagger docs
-

Major functional phases

Phase 1: Project setup and architecture

Goal: create the repo, choose the stack, define the schema, and get the basic backend/frontend running.

You should finish with:

- monorepo created
 - backend boots locally
 - frontend boots locally
 - database connected
 - initial schema designed
 - first migration working
-

Phase 2: Auth and user system

Goal: users can register, log in, and access protected API routes.

You should finish with:

- register route
 - login route
 - hashed passwords
 - JWT auth
 - auth middleware
 - protected test route
 - frontend login/register flow
-

Phase 3: Groups and membership

Goal: users can create groups and belong to groups.

You should finish with:

- create group
 - list my groups
 - view group detail
 - add group member by email
 - permission checks
 - frontend group dashboard
-

Phase 4: Expense ledger

Goal: users can add expenses and split them across members.

You should finish with:

- expenses table
- expense splits table
- create expense API
- database transaction for expense creation
- get group expenses
- frontend add expense form
- frontend expense history

This is one of the most important phases.

Phase 5: Balance calculation and debt optimization

Goal: the backend calculates net balances and optimized settlements.

You should finish with:

- calculate each user's total paid
- calculate each user's total owed
- calculate net balance
- separate debtors and creditors
- generate optimized settlement list
- display results in frontend

This is the project's main non-generic technical feature.

Phase 6: Settlement workflow

Goal: users can mark debts as settled.

You should finish with:

- settlement table
 - create settlement route
 - update balances after settlement
 - settlement history
 - frontend "Settle up" action
-

Phase 7: Testing and correctness

Goal: prove the project is not a toy.

You should finish with:

- unit tests for debt minimization
- API integration tests
- auth tests
- expense creation tests
- transaction correctness tests
- test database setup

This phase matters a lot for backend jobs.

Phase 8: Deployment

Goal: make the app publicly demoable.

You should finish with:

- backend deployed to Google Cloud Run
 - frontend deployed to Vercel
 - Neon database connected
 - production environment variables configured
 - deployed app works end-to-end
-

Phase 9: Polish and resume packaging

Goal: make the project easy for employers to understand.

You should finish with:

- strong README
 - architecture diagram
 - database schema diagram
 - screenshots
 - demo video or GIF
 - resume bullets
 - short LinkedIn/GitHub project description
-

Phase 10: Stretch goals

Goal: add differentiators only after everything else works.

Possible additions:

- WebSockets
- Redis Pub/Sub
- multi-currency
- email invitations
- Flutter client

